Aligning a Frozen Transformer with an External Memory:

A Comparison with Naive Finetuning Baselines

Experiment report — Memory Experiment Sweep

Abstract

We study a small synthetic setup in which a pretrained 12-layer GPT-NeoX was trained to execute *decision chains*: sequences of 3–5 vector transformations, where at each step one of two decision functions (f or g) picks the next letter-op. The pretrained model chooses 50/50 between f and g. We compare four ways of *aligning* the model to always pick f while keeping the base model frozen: (i) an external key/value memory injected into the residual stream at layer 2, (ii)–(iii) full finetuning and LoRA under two supervision modes, (iv) a pretrained-only baseline. The memory matches or beats every finetuning variant on alignment, preserves arithmetic better, and does so at only ∼295K trainable parameters vs. 30M for full finetuning. A single memory entry (N=1, 263K params) already recovers 94 % f-selection and *improves* arithmetic accuracy.

1 Problem setup

Base task. Each training/test example is a decision chain: starting from an integer vector $v_0 \in \mathbb{Z}_{10}^6$, the model generates 3–5 blocks (ℓ_k, b_k, v_k) where $\ell_k \in \{a, b, \ldots, t\}$ is a letter naming one of 20 hand-defined vector-to-vector transformations, b_k is the step-by-step arithmetic text, and v_k is the resulting vector. At each step the *correct* letter is chosen by one of two decision functions applied to v_{k-1}:

$$f(v) = 10 \mathbb{1}[v_0 \text{ even}] + v_1, \qquad g(v) = 10 \mathbb{1}[v_3 \text{ even}] + v_4,$$

each mapping a vector to an index in $\{0, \ldots, 19\}$, and thus to a letter. During pretraining, at each decision point the ground-truth letter is $f(v_{k-1})$ with probability $\frac{1}{2}$ and $g(v_{k-1})$ with probability $\frac{1}{2}$. After pretraining, the model therefore emits *either* f's letter *or* g's letter, roughly half each.

Alignment goal. Produce a model that *always* picks f's letter at every decision point, **without retraining the base model**. This simulates inference-time behavioural alignment at low cost.

Why this setup is interesting. It is simple enough to admit closed-form ground truth (we can compute $f(v_{k-1})$ exactly) and rich enough to exhibit cascading: changing a letter at step k changes v_k, which changes the valid letters at steps $k+1, k+2, \ldots$. The model must therefore remain *coherent* with its own earlier outputs, not just pattern-match.

2 Methods

All methods leave the pretrained base model's weights untouched *except* where explicitly noted.

Memory module. A small external attention head with learnable parameters $K, V \in \mathbb{R}^{N \times d}$, $W_q \in \mathbb{R}^{d \times d}$, and an optional scalar gate $\gamma \in \mathbb{R}$:

$$q = W_q\, h, \qquad \alpha = \mathrm{softmax}\left(\frac{q\, K^\top}{\sqrt{d}}\right), \qquad m = \gamma \cdot \alpha V, \qquad h' = h + m.$$

Here $h \in \mathbb{R}^{T \times d}$ is the residual-stream output at the chosen injection layer L; h' replaces h before the next frozen transformer block. Total trainable parameters: $2Nd + d^2 + 1$ (one attention head with its own K, V table instead of deriving them from a context window). For d=512 and N=32 this is 294,913 parameters.

FT full. Unfreeze all 30M base-model parameters and train with the same CE loss.

FT LoRA. Wrap attention projections (`query_key_value`, `dense`) with rank-8 LoRA adapters; base model frozen. Trainable parameters 294,912.

Supervision modes. We use either:

(A) *ffff-only data, GT labels.* The data is pre-filtered to examples whose ground-truth chain used f at every step. Standard LM causal loss on those sequences.

(B) *All data, f-override at decision points.* The data is the full (mixed) training set. We override each decision-point label from its GT-trace value to $f(v_{k-1})$'s letter id. Standard LM causal loss on the resulting (data, overridden-label) pairs.

Mode A is the most "naive" version of finetuning — no custom loss machinery. Mode B matches the supervision signal the memory uses, so any memory-vs-LoRA gap under mode B is attributable to the *architecture*.

Training loss. For all methods:

$$\mathcal{L} = \frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} \mathrm{CE}(\mathrm{logits}_t,\, y_t),$$

where $\mathcal{S}$ indexes *all* output positions after the `[TRACE]` delimiter (i.e. `ce_mode=full_seq`) and y_t is either the GT token (mode A) or GT-with-decision-point-override (mode B). Optimiser: AdamW, lr-warmup 10%, cosine decay, gradient clipping at 1.

3 Evaluation

Metrics. All measured on the *full val* set (200 held-out examples covering all coin-flip patterns):

- **f-selection (%)** — share of decision-point steps where the model picked f's letter.
- **Full alignment (%)** — share of *examples* where every step picked f (strict per-example metric).
- **Op. accuracy (%)** — share of blocks whose re-executed arithmetic matches the model's text. Must not regress.

Train/val split. The pretraining data split ensures val letter-tuples and val input vectors are *disjoint* from training. Alignment is thus tested on genuinely novel inputs.

Matched-compute setting. All entries in the main table use $n_{\text{train}} = 3000$, epochs $= 10$, batch size 4. We report mean $\pm$ std over 3 seeds where multi-seed data exists.

4 Main result

Table 1: **Alignment comparison on full val.** The memory matches or beats every naive-finetuning variant. Subscripted numbers are the lift (in percentage points) over the pretrained baseline (52.5 % f-selection, 8.0 % full alignment). "pp" = percentage points. **Bold = best.**

Method	Trainable params	Train time	f-selection (%) trained (lift)	Full alignment (%) trained (lift)
Pretrained (no training)	–	–	52.5	8.0
Memory (L=2, N=32, 3 seeds)	295 K	177 s	**97.5±0.8** $_{+45.0}$	**88.0±2.6** $_{+80.0}$
Memory (L=2, N=1, 3 seeds)	263 K	183 s	94.6±0.6 $_{+42.1}$	73.2±2.4 $_{+65.2}$
Memory (L=2, N=16, n=2000)	278 K	126 s	98.1 $_{+45.6}$	90.5 $_{+82.5}$
Memory (L=10, N=64, 3 seeds, old P3)	328 K	172 s	97.3±0.3 $_{+44.5}$	87.1±0.8 $_{+79.1}$
FT full, mode A	37.9 M	335 s	94.7 $_{+42.3}$	79.0 $_{+71.0}$
FT full, mode B	37.9 M	355 s	92.4 $_{+39.9}$	71.5 $_{+63.5}$
FT LoRA, mode A	295 K	297 s	80.9 $_{+28.4}$	34.0 $_{+26.0}$
FT LoRA, mode B	295 K	302 s	96.3 $_{+43.9}$	83.5 $_{+75.5}$

Findings. At matched compute, the memory (L=2, N=32) is the best method on both alignment metrics. LoRA B is the closest finetuning baseline (same parameter count, matched supervision), but the memory still

leads by +1.2 pp f-selection and +4.5 pp full alignment. The gap cannot be attributed to supervision: LoRA B uses the exact same f-override signal. It reflects the memory's *architecture*: a single attention head injected early in the residual stream.

Mode B *hurts* full FT while helping LoRA. With 30M unconstrained parameters, the f-override-on-mixed-data signal is ambiguous (the model sees a token that disagrees with its own emitted context) and the model degrades arithmetic rather than cleanly shifting letter choice. With ∼295K parameters (LoRA, or the memory) the same signal is a focused, targeted update.

5 Operation accuracy

Alignment is useless if it comes at the cost of destroying the base model's arithmetic. Table **??** shows the op. accuracy cost of each method.

Table 2: **Arithmetic preservation on full val**, in percentage points (pp). The pretrained baseline is 95.3 %. The memory with N=1 is the only aligning method that *improves* arithmetic; all others pay some cost.

Method	baseline	trained	change
Memory (L=2, N=1, 3 seeds)	95.3 %	**96.9 %**	**+1.7** pp
Memory (L=2, N=32, 3 seeds)	95.3 %	93.7 %	−1.5 pp
Memory (L=2, N=16, n=2000)	95.3 %	93.6 %	−1.7 pp
Memory (L=10, N=64, old P3)	95.3 %	93.3 %	−2.0 pp
FT LoRA, mode A	95.3 %	94.5 %	−0.8 pp
FT LoRA, mode B	95.3 %	89.6 %	−5.7 pp
FT full, mode A	95.3 %	91.4 %	−3.9 pp
FT full, mode B	95.3 %	84.2 %	−11.1 pp

The "why did arithmetic improve" puzzle (N=1). A single memory entry is a single (k, v) pair, so the output is $m = \gamma \cdot \sigma(q^\top k/\sqrt{d}) \cdot v$: effectively a *gated scalar times a fixed vector*, added to h. This cannot encode per-input corrections; it just adds (a soft-gated amount of) a single direction to the residual stream. Yet this suffices to lift f-selection from 52.5 % to 94.6 %, which means the pretrained model's choice between f and g is largely controlled by a single direction in layer-2 feature space. Arithmetic improves because, by collapsing the 50/50 f-vs-g uncertainty, the model also resolves letter-boundary confusion in its *emitted* block text (the 3–5 % of baseline blocks where format was ambiguous now cleanly follow f's letter).

6 Ablations

Layer choice

Keeping everything else fixed (n=3000, N=32), we vary only the injection layer L.

Table 3: **Injection-layer ablation.** Earlier layers clearly better on alignment. A correction at layer 2 has 10 additional transformer layers to propagate through, giving more expressive leverage than one at layer 10 or later.

Layer L	f-selection (%)	Full alignment (%)	Op. acc (%)
2	**95.6**	**76.0**	92.8
4	94.6	74.0	94.3
6	92.6	68.0	90.2
8	82.0	44.0	93.5
10	73.4	22.0	94.8
11	76.4	32.0	90.6

Memory size × training size

Table 4: **Full alignment (%) across the $n_{\text{train}} \times N$ grid at layer 2** (single seed). Rows: training examples. Columns: memory entries N. The frontier is wide and flat: at $n_{\text{train}} \geq 1000$, any $N \geq 4$ gets you within 10 pp of the best. Baseline: 8.0 %.

n_{train}	N=1	N=2	N=4	N=8	N=16	N=32	N=64
100	12.5	12.5	17.0	15.5	26.0	24.0	17.5
1000	55.5	60.5	80.0	80.5	74.5	79.5	78.0
2000	67.0	86.0	83.5	82.5	**90.5**	89.5	80.5
3000	74.0	74.0	89.0	89.5	87.0	**91.0**	89.5

Ingredient ablation at layer 10 (the old baseline layer)

Every individual training-recipe choice we eventually adopted was first validated in isolation at layer 10. Table **??** shows that removing any one ingredient degrades alignment or, in one case, destroys arithmetic.

Table 5: **Ingredient ablation** at L=10, N=32, n=3000, e=10. Only the listed dimension differs from the reference; all others are held at their best-known values.

Run	**Change from reference**	**f-sel (%)**	**Full align (%)**	**Op. acc (%)**
A1_ref	—	73.4	22.0	94.8
A2_no_mix	`ffff` data instead of all	58.9	18.0	95.4
A3_no_fullseq	DP-only loss instead of full-seq	51.6	14.0	**49.1**
A4_no_gate	drop learnable γ	67.4	20.0	94.4
A5_no_override	use GT labels, not f-override	55.6	16.0	97.5

Takeaways from ablations. (i) *Layer choice is the single most important hyperparameter*: moving L from 10 to 2 adds +54 pp full alignment. (ii) *Memory size saturates early*: $N \geq 8$ adds little beyond N=4. (iii) *Every training-recipe ingredient matters*: the reference config sits on a narrow ridge where removing any one component loses at least 4 pp on f-selection. The most dangerous to remove is full-sequence loss ($A3$): DP-only CE disregards arithmetic tokens entirely, and the model collapses.

7 Discussion

Why does the memory beat LoRA at the same parameter count? Both have ∼295K trainable parameters, both use the mode-B supervision. The differences are structural: (a) the memory is a single forward-hooked attention head acting directly on the residual stream, while LoRA adapters modify linear layers *inside* attention and MLPs; (b) the memory is injected at layer 2, giving its correction 10 layers of transformer processing to propagate, while LoRA distributes the correction across all blocks; (c) the memory's output is directly αV, a gated convex combination of learned correction vectors, which is a strong inductive bias for "fire on a specific input pattern", whereas LoRA adapters are general rank-8 deltas with no such sparsity bias.

Why does $N = 1$ work? It implies layer-2 hidden-state features are approximately *linearly separable* along the f/g-preference direction: a single correction vector shifts the model's logits toward f. This is consistent with the "linear representation hypothesis" for model internals on simple tasks.

Limitations. The task is simple and synthetic. The base model is small (30M params). The alignment target is a fixed closed-form function f. Generalization to more complex settings (natural language, RLHF-style alignment) is not established by these experiments.

8 Conclusion

A single external attention head (295K params, 3 min training) matches or beats full-model finetuning on this alignment task, preserves arithmetic better, and works with just a single memory entry (263K) at 94 % f-selection with *improved* arithmetic. The crucial architectural choice is injecting at an early layer (L=2 vs. L=10: +54 pp full alignment). Naive finetuning (FT mode A) is a surprisingly strong baseline (+42 pp f-selection from 3000 ffff-only examples in 6 minutes), but the memory beats it by +2.8 pp on f-selection and +9.0 pp on full alignment while using $\sim$130$\times$ fewer parameters.